Demo6VertexNormal
The Sample Project Mini-Cookbook

This FREE booklet is designed to accompany the recently added new sample project `Demo6VertexNormal` from the book Practical C++ Game Programming with Data Structures and Algorithms. It provides additional insights into the code logic, algorithms, and problem-solving approaches used in the implementation of this sample.

In this mini-booklet, you'll find how the sample project works and the key ideas behind its design.

Before we begin

You will need the knowledge of *Chapter 6* to understand how the sample works, or basic understanding of how lighting in raylib works. Make sure you've finish reading the section *Lighting the world*, before the next section *Achieving better realism*. It's best time to try out this sample before moving to next advanced lighting topic. This booklet will not repeatedly cover anything you need to know already in the book.

Getting started on the sample project

This sample compares the visual differences between using default vertex normal and averaged vertex normal to calculate different 3D lighting effects.

First, make sure you have pulled the latest updates from GitHub (https://github.com/PacktPublishing/Practical-C-Game-Programming-with-Data-Structures-and-Algorithms/tree/main).
Open the Visual Studio solution file included with this book in Visual Studio Community, select the `Demo6VertexNormal` project, rebuild it, and run. You should see the following output:

Figure 1 – demo shows different lighting result on the same 3D model

Both robots use the same 3D model, but you'll notice that the lighting on the left one appears as a smooth surface, while the lighting on the right reveals each individual triangle of the model more realistically.

In this scene, we placed four point-lights. Each of them casts different color. You can toggle them on or off using the *Y*, *R*, *G*, and *B* keys. This makes it easier to observe how the rotating lights create different reflections on the surfaces of the two robots.

So, what causes this difference in lighting effects on two robots loaded from the same 3D model?

The Difference Lighting Effect

In *Chapter 6* of *Practical C++ Game Programming with Data Structures and Algorithms*, we introduced the basics of GPU shaders along with several real-time lighting algorithms. We explained how directional light and point light are calculated in *Understanding directional light* and *Understanding point light*. While these two

types of light differ in some details, both rely primarily on the triangle's vertex or face normal and the direction of the light source to compute lighting intensity.

So, you may have already guessed: since both models are illuminated by the same light sources, the only difference lies in the normal used to calculate the lighting result.

Back in *Chapter 1*, we introduced how Knight loads 3D models. Knight relies on raylib's `LoadModelAnimation()` function to import models. However, raylib's support for the *glTF* format loading is only partial—it supports loading the model as a list of triangles. In practice, though, many of those triangles share common vertices in the original 3D model:

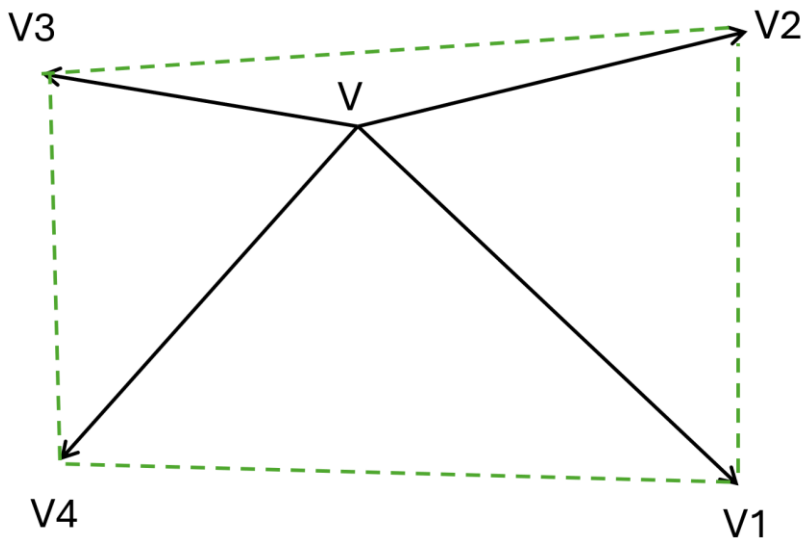

Figure 2 - Four triangles share the same vertex position V.

Imagine a vertex *V* that is shared by several adjacent triangles. Although there might be only one shared vertex in the actual model, in the data structure of triangle list, each triangle is stored independently, so there are actually 4 copies of duplicate vertex *V*. As a result, the shared vertex V is not treated as a single vertex, but rather as four separate vertices at the same position: one in each of the adjacent triangles [*V, V4, V1*], [*V, V1, V2*], [*V, V2, V3*], and [*V, V3, V4*].

Moreover, these four duplicate *V* vertices each have different normal vector. For example, in the triangle [*V, V1, V4*], the normal of *V* (shown in orange as Normal *V1–V4*) is different from the normal of *V* in the triangle [*V, V1, V2*] (shown in green as Normal *V1–V2*), as illustrated in *Figure 3*.

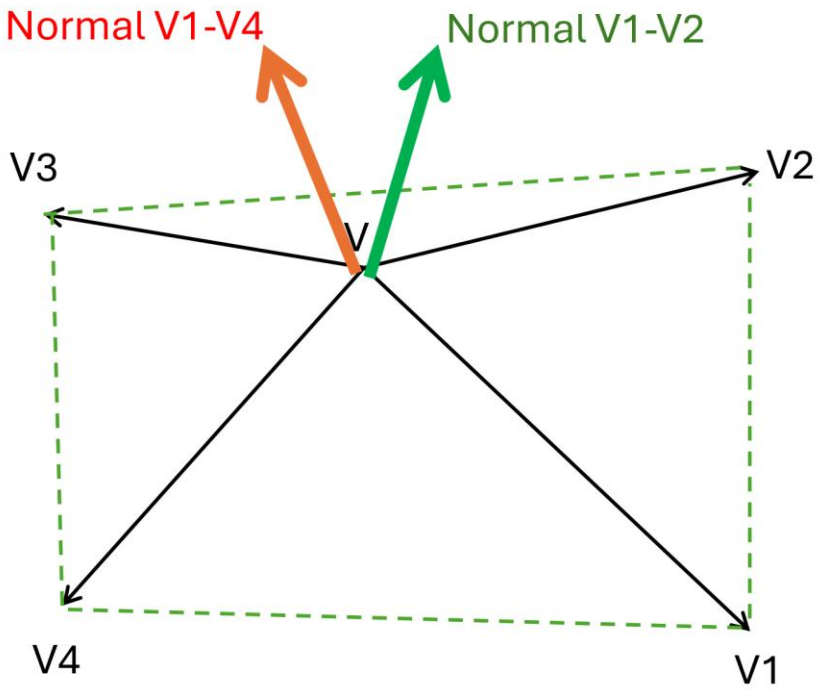

Figure 3 – Different triangles produce different normal of vertex shared with other triangles.

The green and orange normal vectors in *Figure 3* show that two adjacent triangles assign different normal to the same shared vertex at position *V*.

This happens because when the original model exported as a list of triangles, the normal of a vertex depends on the triangle it belongs to. For example, the normal of vertex V in triangle [*V, V4, V1*] is calculated using the cross product of vectors (*V4 – V*) and (*V1 – V*) – the same as face normal. Meanwhile, the normal of *V* in triangle [*V, V1, V2*] is derived from different cross product results from different triangles, and thus points in a

different direction. Since each triangle has a distinct orientation, the resulting normal vector at the same shared vertex position are not identical. That's why each triangle duplicates the same shared vertex *V* with its own normal vector.

The normal vectors loaded by raylib's `LoadModelAnimation()` function come directly from the 3D modeling software's exported data. In the original 3D model, shared vertices across different triangles already have inconsistent normal vectors. Sometimes, the model data doesn't even contain any normal data and we need to calculate by ourselves.

Therefore, if we simply load and render the model as-is, we get the result shown in the demo's right-hand robot: a faceted surface where the edges between triangles appear sharp and angular.

Producing Smooth Lighting Effects

In many cases, we would prefer smoother shading—even when using relatively few triangles—so that the surface appears smoother and curvier.

Lighting is determined by the angle between the light source and the surface normal. Therefore, to achieve a smooth effect, all triangles that share a vertex must also share the same normal at that vertex. Besides, the same normal should be the normal averaged by face normal of all adjacent triangles:

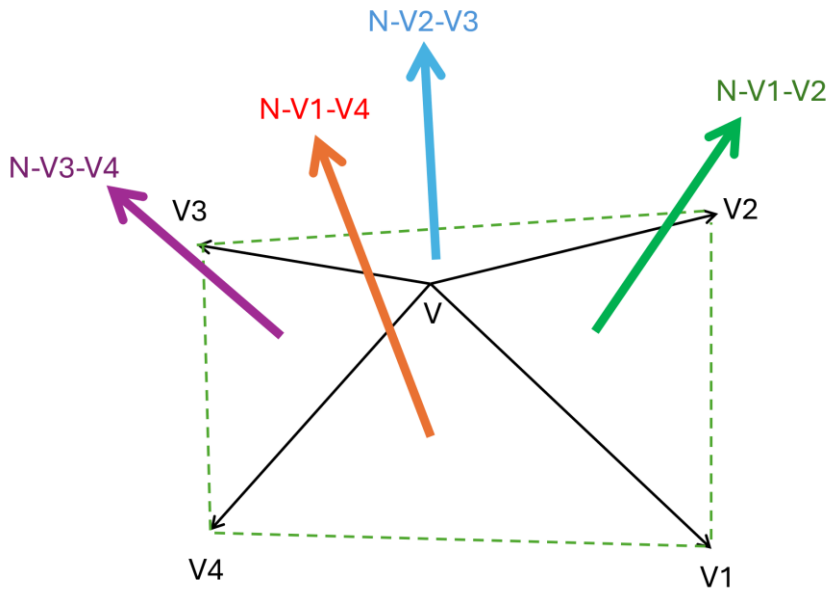

Figure 4 – We need to calculate average of face normal vectors with all triangles shared the same vertex

Like *Figure 4*, suppose vertex *V* is shared by four triangles. If the normal vectors of the triangle face are *N-V1V4*, *N-V1V2*, *N-V2V3*, and *N-V3V4*, then the smoothest result comes from averaging those four adjacent face normal vectors to compute a new, unified normal for V:

To achieve this, we need to recalculate the normal vectors in each `Mesh` of the raylib `Model` after it has been loaded. This computation is implemented in Knight's `RecalculateSmoothNormals()` function, a utility function located in `Utils.cpp` of the Knight project.

Now let's look at `RecalculateSmoothNormals()` function. For every `Mesh` in the Model, we recalculate its normal vectors one by one:

```
for (int midx = 0; midx < model.meshCount; ++midx)
{
  //Calculate normal of vertices in each mesh …
}
```

For each mesh, the process works as follows:

1. Enumerate all the triangles in the `Mesh` and identify the three vertices (V0, V1, V2) that form each triangle.
2. Compute the face normal of the triangle.
3. Add this face normal to a hash map of shared vertices. Here, we use the standard C++ `map` data structure to quickly match vertices that occupy the same position.

```
std::map<Vector3, Vector3, Vector3Compare> accumulatedNormals;
```

The core computation loop is as follows:

```
for (int i = 0; i < mesh.vertexCount; i += 3) {

 Vector3 v0 = { mesh.vertices[i * 3], mesh.vertices[i * 3 + 1],
mesh.vertices[i * 3 + 2] };

 Vector3 v1 = { mesh.vertices[(i + 1) * 3], mesh.vertices[(i + 1)
* 3 + 1], mesh.vertices[(i + 1) * 3 + 2] };

 Vector3 v2 = { mesh.vertices[(i + 2) * 3], mesh.vertices[(i + 2)
* 3 + 1], mesh.vertices[(i + 2) * 3 + 2] };

 Vector3 faceNormal = Vector3Normalize(Vector3CrossProduct(

 Vector3Subtract(v1, v0), Vector3Subtract(v2, v0)));

 accumulatedNormals[v0] = Vector3Add(accumulatedNormals[v0],
faceNormal);

 accumulatedNormals[v1] = Vector3Add(accumulatedNormals[v1],
faceNormal);

 accumulatedNormals[v2] = Vector3Add(accumulatedNormals[v2],
faceNormal);

}
```

In the code snippet:

- First, we identify the three vertices `V0`, `V1`, and `V2` that make up a triangle.
- Next, we calculate the face normal using `V0`, `V1`, and `V2`.

- Then, we add this face normal to the list of normal vectors for any vertices that are shared with other triangles.

After this, we iterate over all the vertices in the mesh. For each vertex, we look it up in `accumulateNormals`, average the face normal vectors of all adjacent triangles, and write the result back as the new vertex normal for that shared vertex.

```
for (int i = 0; i < mesh.vertexCount; ++i) {

 Vector3 pos = { mesh.vertices[i * 3], mesh.vertices[i * 3 + 1],
mesh.vertices[i * 3 + 2] };

 Vector3 normal = Vector3Normalize(accumulatedNormals[pos]);

 mesh.normals[i * 3] = normal.x;

 mesh.normals[i * 3 + 1] = normal.y;

 mesh.normals[i * 3 + 2] = normal.z;

}
```

With this, we complete the calculation of new smoothed vertex normal vectors across adjacent faces like *Figure 5*. Finally, we use `UpdateMeshBuffer()` to upload the recalculated mesh normal vectors back to the GPU.

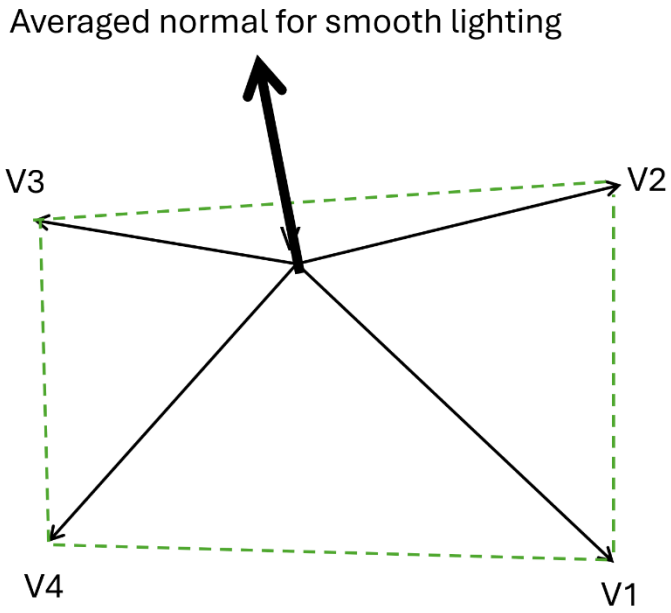

Figure 5 – The final averaged normal vector of shared vertex position V

After completing the utility function for recalculating normal vectors, we added a new variable in `ModelComponent.h`:

```
bool _AlwaysSmoothNormal = true; // If true, recalculate smooth
vertex normals for the model
```

By default, all imported 3D models will have their normal vectors recalculated. If you want to preserve the original normal vectors from the 3D model, you can set `_AlwaysSmoothNormal` to false before calling `ModelComponent.Load3DModel()` like how we load the right side robot in the `Start()` function of `Demo6VertexNormal.cpp`:

```
ModelComponent* FNPlayerModel = FNActor-> CreateAndAddComponent
<ModelComponent>();

FNPlayerModel->_AlwaysSmoothNormal = false;

FNPlayerModel->Load3Dmodel
("../../resources/models/gltf/robot.glb");
```

So, when the program runs, the 3D model of `VNActor` on the left appears smooth, while the 3D model of `FNActor` retains its original faceted look.

Conclusion

This sample demonstrates how to recalculate normal vectors loaded by raylib to achieve smooth shading, or alternatively, leave the original per-triangle normal vectors to produce a faceted lighting effect.

It's important to note that this is NOT a bug in raylib—it simply reflects different shading requirements. The sample shows how recalculating normal vectors can produce very different visual results from the same 3D model.

Challenge Yourself

For simplicity, this sample project uses the `_AlwaysSmoothNormal` in the `ModelComponent` class to control whether the normal of all meshes in the 3D model are recalculated.

In practice, whether smooth shading is applied should be determined at the `Material` property, on a per-`Mesh` basis.

Although raylib's `Material` does not provide a built-in setting for this, its data structure includes `params[4]` fields that applications can use to define custom requirements.

```
// Material, includes shader and maps
typedef struct Material {
    Shader shader;          // Material shader
    MaterialMap *maps;      // Material maps array
(MAX_MATERIAL_MAPS)
    float params[4];        // Material generic parameters (if
required)
} Material;
```

In raylib's `Model`, each `Mesh` has a corresponding `Material`. As a challenge, try implementing support for a per-mesh `_AlwaysSmoothNormal` setting without modifying the underlying raylib Material or Model structures.

This would allow for more nuanced control, such as making the robot's head appear smooth and rounded, while keeping the body with its original blocky look.

Lighting in realtime 3D graphics is fun! Isn't it? Simple changes of manipulating normal vector, you can explore different 3D rendering effects!

Happy coding and learning!